

Multiple Models for the NLI task

Zihan Wu

**University of Western Australia
Perth, Australia**

24372276@student.uwa.edu.au

Jianyun Yang

**University of Western Australia
Perth, Australia**

24181422@student.uwa.edu.au

Haoran Yu

**University of Western Australia
Perth, Australia**

24180266@student.uwa.edu.au

Abstract

Natural Language Inference (NLI) is a fundamental task in natural language understanding that aims to determine the logical relationship between a pair of sentences — typically identifying whether a hypothesis is entailed, contradicted, or neutral with respect to a premise. This study investigates and compares three neural architectures for the NLI task: a Bidirectional Long Short-Term Memory (BiLSTM) model, an Encoder–Decoder model with an LSTM encoder and GRU decoder, and a Transformer-based model. The BiLSTM serves as a baseline for sequential modeling, while the Encoder–Decoder model explores cross-sequence representation learning, and the Transformer model leverages global attention mechanisms for contextual understanding. Experiments were conducted on a benchmark NLI dataset to evaluate classification performance and generalization ability. The results show that the Transformer-based model achieves the highest overall accuracy, confirming its superior ability to model long-range dependencies and semantic nuances. The LSTM–GRU encoder–decoder model demonstrates moderate improvement over BiLSTM, indicating that hybrid recurrent structures with attention can enhance representation learning. These findings highlight the progressive effectiveness of attention mechanisms and non-recurrent architectures for natural language reasoning tasks.

1 Introduction

Natural Language Inference (NLI), also known as Recognizing Textual Entailment (RTE), is a core problem in natural language understanding. The task involves determining whether a given hypothesis can be logically inferred from a premise sentence, or whether the relationship between them is one of entailment, contradiction, or neutrality. NLI plays an essential role in applications such as question answering, dialogue systems, information retrieval, and text summarization, where un-

derstanding sentence-level meaning and reasoning is crucial.

Early NLI systems relied heavily on hand-crafted linguistic features, syntactic rules, or lexical resources. However, such approaches often struggled to capture the subtleties of context and semantics in natural language. With the rise of deep learning, neural network models have become the dominant approach, capable of automatically learning distributed representations of words and sentences. Despite this progress, the choice of architecture significantly influences model performance, as different networks capture context and dependencies in distinct ways. And there are several models that are commonly used in the NLI tasks, which are RNN-based model and Transformer-based models.

This paper presents a comparative analysis of three representative neural architectures for the NLI task: a Bidirectional LSTM (BiLSTM), an Encoder–Decoder model combining an LSTM encoder and GRU decoder, and a Transformer-based model. The BiLSTM provides a sequential baseline that processes input in both temporal directions, capturing local context efficiently. The LSTM–GRU Encoder–Decoder model introduces a hierarchical structure that can learn complex inter-sentence representations through sequence-to-sequence processing. Finally, the Transformer model applies multi-head self-attention to capture global dependencies without recurrence, offering the potential for improved reasoning over longer contexts.

Compared to existing methods of multiple papers, we have applied some updates into our models. The innovation of the BiLSTM model lies in the integration of multiple attention mechanisms to explore how different attention strategies influence the model’s ability to align and interpret semantic relations between the premise and hypothesis. This design enables a deeper understanding of which contextual cues are most critical for entailment

recognition. The LSTM–GRU Encoder–Decoder model introduces the GRU as the decoder to reduce the computational overhead of traditional LSTM-based decoders while maintaining competitive representational capacity. This hybrid architecture allows efficient gradient propagation and faster convergence, making it suitable for NLI tasks with large sentence pairs. Lastly, the Transformer-based model departs from conventional encoder-focused designs by employing the Transformer architecture in the decoder stage. This modification allows the model to enhance interpretive reasoning in the final inference stage, leveraging attention-based decoding to better integrate semantic relationships between encoded premise–hypothesis representations.

2 Methods

2.1 Model A—BiLSTM encoding only

In this model, we use BiLSTM encoding only to train the dataset. BiLSTM is Bidirectional Long Short-Term Memory model which considers the forward stream of data and backward stream of data at the same time.

Architecture The input part for this model is the embedded vector. Suppose the embedded vector of premise and hypothesis are

$$\mathbf{E}_P = [\mathbf{e}_{p_1}, \mathbf{e}_{p_2}, \dots, \mathbf{e}_{p_m}] \in R^{m \times 100} \quad (1)$$

$$\mathbf{E}_H = [\mathbf{e}_{h_1}, \mathbf{e}_{h_2}, \dots, \mathbf{e}_{h_n}] \in R^{n \times 100} \quad (2)$$

Then, we use LSTM to encode vector with considering the hidden state from forward and backward

$$\vec{\mathbf{h}}_t = \text{LSTM}(\mathbf{e}_{p_t}, \vec{\mathbf{h}}_{t-1}) \quad (3)$$

$$\overleftarrow{\mathbf{h}}_t = \text{LSTM}(\mathbf{e}_{p_t}, \overleftarrow{\mathbf{h}}_{t+1}) \quad (4)$$

The hidden state at position t is the concatenation of both directions:

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in R^{256} \quad (5)$$

The complete BiLSTM outputs are:

$$\mathbf{O}_P = [h_1, h_2, \dots, h_m] \in R^{m \times 256} \quad (6)$$

$$\mathbf{O}_H = [h_1, h_2, \dots, h_n] \in R^{n \times 256} \quad (7)$$

Then, we applied the attention mechanism. The attention mechanism assigns weights to each hidden state to highlight the most relevant parts of the

sequence. For each time step t , the attention score s_t is computed using a linear transformation:

$$\mathbf{s}_t = W_a \mathbf{h}_t + b_a \quad (8)$$

where $W_a \in R^{256 \times 1}$ and $b_a \in R$.

Attention weights α_t are obtained via the softmax normalization:

$$\alpha_t = \frac{\exp(s_t)}{\sum_{t'=1}^m \exp(s_{t'})} \quad (9)$$

The attended representation of the premise and the attended representation of the hypothesis should be

$$\mathbf{a}_P = \sum_{t=1}^m \alpha_t \mathbf{h}_t \in R^{256} \quad (10)$$

$$\mathbf{a}_H = \sum_{t=1}^n \beta_t \mathbf{h}_t \in R^{256} \quad (11)$$

The attended vectors of the premise and hypothesis are concatenated and passed through a fully connected layer to predict the relationship label and The output logit o is converted into a probability using the sigmoid function

$$\mathbf{c} = [a_P; a_H] \in R^{512} \quad (12)$$

$$\mathbf{o} = W_f \mathbf{c} + b_f \quad (13)$$

$$\mathbf{p} = \sigma(o) = \frac{1}{1 + e^{-o}} \quad (14)$$

2.2 Model B: Encoder–Decoder with Cross-Attention

Model B is designed to capture complex relationships between premise and hypothesis using an encoder–decoder framework with cross-attention. The encoder is a single-layer bidirectional LSTM (BiLSTM) that encodes the premise. The decoder is a single-layer GRU that generates the hypothesis representation, attending to the encoded premise at each step via dot-product cross-attention.

Architecture Let $P = (p_1, \dots, p_{n_p})$ be the premise token sequence and $H = (h_1, \dots, h_{n_h})$ the hypothesis. The premise is embedded and encoded as $\mathbf{E}_P = \text{BiLSTM}(\text{Embed}(P))$. The decoder GRU processes the hypothesis tokens, and at each step t , computes attention weights $\alpha_t = \text{softmax}(\mathbf{E}_P \cdot \mathbf{q}_t)$, where $\mathbf{q}_t$ is the decoder hidden state. The context vector $\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{E}_{P,i}$ is concatenated with the decoder input.

Pooling Strategies To obtain a fixed-size sentence representation, we ablate four pooling methods over the encoder outputs: mean pooling, max pooling, last hidden state, and a learned query vector (attention pooling). The pooled vector is used for final classification.

Equations

$$\mathbf{E}_P = \text{BiLSTM}(\text{Embed}(P)) \quad (15)$$

$$\mathbf{q}_t = \text{GRU}(h_{t-1}, y_{t-1}) \quad (16)$$

$$\alpha_t = \text{softmax}(\mathbf{E}_P \cdot \mathbf{q}_t) \quad (17)$$

$$\mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{E}_{P,i} \quad (18)$$

2.3 Model C: Decode-only Transformer with Disentangled Self-Attention

Architecture Overview We concatenate the premise and hypothesis and pass them through a decoder-only Transformer stack with a disentangled self-attention module that separates content (token embeddings) from position (sinusoidal features). Instead of adding positions to tokens, we keep them separate and learn distinct projections for content vs. position. After the stack, we split the sequence back into premise/hypothesis spans, apply masked mean pooling over each span, combine the two vectors with standard NLI matching features, and classify with a small MLP.

Disentangled Multi-Head Self-Attention Gaining insight from this paper (He et al., 2020), attention logits are the sum of three terms instead of one content $\leftrightarrow$ content score:

$$\begin{aligned} \mathbf{c2c}: & \text{content} \rightarrow \text{content} & \mathbf{c2p}: & \text{content} \rightarrow \text{position} \\ \mathbf{p2c}: & \text{position} \rightarrow \text{content} \end{aligned}$$

Values still come from content. Position is never added to token embeddings, it only influences how attention is aimed.

$$\mathbf{q}_t^{(c)} = W_Q^{(c)} \mathbf{x}_t, \quad \mathbf{k}_t^{(c)} = W_K^{(c)} \mathbf{x}_t, \quad \mathbf{v}_t^{(c)} = W_V^{(c)} \mathbf{x}_t, \quad (19)$$

$$\mathbf{q}_t^{(p)} = W_Q^{(p)} \mathbf{r}_t, \quad \mathbf{k}_t^{(p)} = W_K^{(p)} \mathbf{r}_t. \quad (20)$$

The attention score from position i (query) to j (key) is

$$s_{ij} = \frac{\langle \mathbf{q}_i^{(c)}, \mathbf{k}_j^{(c)} \rangle + \langle \mathbf{q}_i^{(c)}, \mathbf{k}_j^{(p)} \rangle + \langle \mathbf{q}_i^{(p)}, \mathbf{k}_j^{(c)} \rangle}{\sqrt{d_h}} \quad (21)$$

The padding mask prevents attention to placeholder PAD tokens. We then apply the softmax function to the masked scores to convert them into a valid probability distribution, which represents as the attention weights (A). We then apply a linear projection, dropout, and the usual residual-norm and FFN sublayers per Transformer block.

Positional Features For time step t (0-indexed), the sinusoidal positional vector $\mathbf{r}_t \in \mathbb{R}^d$ uses the standard $\sin / \cos$ basis. We apply dropout to $\{\mathbf{r}_t\}$ and do not add it to token embeddings. **Span Pooling and NLI Head** After the stack, split the sequence back into premise and hypothesis spans. Do masked mean-pooling on each span to get two vectors. Build $[p, h, \text{lp-hl}, [p \odot h]]$ and pass through a small MLP to get a single logit, then probability.

Justification for the Design *Disentangling content and position*: it helps separate token semantics from positional information, which allows the model to capture lexical relationships and the relative placement of evidence. *Decoder-only*: it lets the hypothesis attend to earlier premise tokens within one stream, which simplify implementation and improving efficiency.

3 Experiment Setup

3.1 Dataset Description

The dataset is divided into three splits:

- **Training split** (train.json): Contains 23,088 examples (indices 0–23,087), with approximately 45% “entails” and 55% “neutral” labels (rough estimate based on the excerpt).
- **Validation split** (validation.json): Contains 1,304 examples (indices 0–1,303), with roughly 48% “entails” and 52% “neutral” labels.
- **Overall distribution**: About 46% “entails” and 54% “neutral” across the dataset.

Split	Examples	Labels (% E/N)
Train	23,088	45 / 55
Validation	1,304	48 / 52
Test	2,126	46 / 54

Split	Min Tokens	Max Tokens
Train	4	30
Validation	16	44
Test	2	41

Table 1: Dataset statistics and token counts for training, validation, and test splits.

3.2 Implementation Details

Model Configuration For model A and B, we applied the same configuration parameters to them. We set the hidden size to 128 and embedding dimension to 100.

Hyperparameter	Value
embedding dimension	100
Hidden dimension	128
Dropout rate	0.2
Pooling types	mean, max, last, attention
Learning rate	1×10^{-3}
Batch size	32
Epochs	3
Patience	2
Max sequence length	50

Table 2: Model A and B configuration and hyperparameters.

For model C-Transformer: $N=4$ decoder layers, $d_{\text{model}}=256$, $n_{\text{heads}}=8$, FFN hidden = 512. Disentangled attention dropout 0.2; sinusoidal feature dropout 0.2. Classifier: MLP $4d \rightarrow d \rightarrow 1$ with ReLU and dropout 0.2. Masks: causal over the concatenated sequence; key padding mask for pads. Pooling: masked mean per span. Training-Loss: BCEWithLogitsLoss. Optimizer: Adam, LR 1×10^{-4} . batch size: 32. Train up to 10 epochs with early stopping; patience: 2 on validation accuracy.

4 Result

4.1 Main Performance Comparison

In this part, we compare the accuracy score of these three models in the test data set. The outcome shows that for model A, B, and C, the accuracy score are 0.69285, 0.7262, 0.76783

Model	Accuracy
BILSTM	0.69285
BILSTM-GRU	0.7262
Transformer-based	0.76783

Table 3: Accuracy for different model.

This result fits for what we expected, since The model C - transformer-based model achieves the best performance. And the reason why Transformer-based model achieves the highest accuracy is because its self-attention mechanism captures global contextual dependencies more effectively than sequential models like BiLSTM and BiLSTM-GRU, enabling richer feature representation and better generalization.

4.2 Ablation study for attention

In this part, we applied four different attention format for BILSTM model to test which one is the best, which are Self-attention, dot-attention, general-attention and additive-attention. For more math details, refer to **Appendix A**

Self-Attention. Each word in the sentence “at-tends” to itself. the model learns which tokens within the same sentence are more important when forming a sentence-level representation.

Dot-Product Attention. Measures interaction between two sequences (the premise and hypothesis) using a simple dot product. It tells the model how well each token in the premise aligns with each token in the hypothesis.

General Attention. A generalization of dot-product attention that includes a learnable linear transformation before computing similarity.

Additive (Bahdanau) Attention. Instead of using dot products, it learns a small feed-forward neural network to compute attention scores. Additive attention learns nonlinear relationships between tokens in the premise and hypothesis, rather than just relying on dot products.

Attention Type	Accuracy
Self	0.681
Dot	0.697
General	0.692
Additive	0.705

Table 4: different Attention type accuracy comparison

We also study how cross content-position interactions shape performance by ablating the c2p and p2c in the disentangled attention logits on model C (variants equation refers to Appendix B). The result (table 5) shows that full model has the best performance on accuracy (0.767), followed by c2p_p2c (0.762), c2p_only (0.754), and p2c_only (0.680). Overall, the full model achieves the highest test accuracy but tend to train longer. Both c2p_only and c2p_p2c train faster, with a acceptable drop in accuracy. The result also suggests that c2p is more useful than p2c: the p2c_only variant performs worst, while c2p_only remains as a competitive model.

Table 5: Test accuracy with deltas vs. full

Mode	Test Acc	Δ Test vs. Full
full	0.767	+0.0000
c2p_only	0.754	−0.0122
p2c_only	0.680	−0.0870
c2p_p2c	0.762	−0.0042

4.3 Additional Ablation study

In this part, we did a Additional Ablation study for BiLSTM model by changing the hidden size from 64 o 256, which is we used three values as a hidden size list [64,128,256]. In this part, we combined different attention mechanisms and different hidden size, then calculate the average accuracy of diffrent attention mechanisms. The outcome shows when hidden layer is 256, it will get the highest accuracy, which is 0.7135

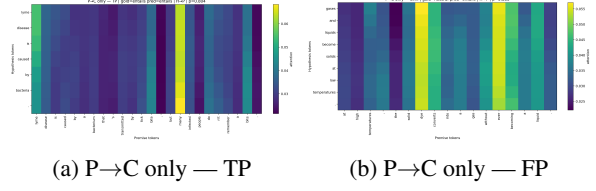
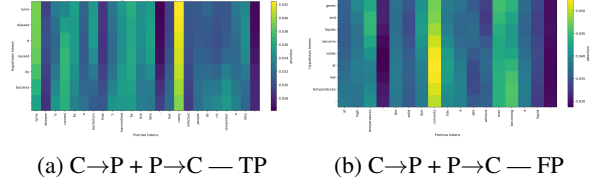
Hidden Size	Accuracy
64	0.681
128	0.694
256	0.7135

Table 6: different hidden size accuracy comparison

That makes sense since when hidden layer becomes bigger, it will catch more possible features of the model and make it more accurate when other parameters keep same.

5 Qualitative Study

We select one test-set example for qualitative analysis on model C and extract attention weights for a true positive (TP) and a false positive (FP). The results(here just display two variants, other two refers to appendix C) show that all variants concentrate hypothesis→premise ($H \rightarrow P$) attention on a small set of premise columns, and yield high-confidence correct decisions on entailment cases. In neutral cases with high surface overlap but different logic (e.g., negation or conditionals), attention clings to overlapping words and underweights logical composition, which leads to false-positive predictions. Models that include cross terms—especially $C \rightarrow P + P \rightarrow C$ —tend to be more conservative on FPs and help reduce overconfidence.

Figure 1: $P \rightarrow C$ only model: TP vs. FP.Figure 2: $C \rightarrow P + P \rightarrow C$: TP vs. FP.

6 Conclusion

In this project, we have built three models with different architectures: BiLSTM, an LSTM-GRU encoder-decoder, and a decoder-only Transformer with disentangled self-attention. The Transformer-based model has the highest test accuracy (0.7678). The attention ablation studies indicate that additive attention performed best (0.705), and additional ablation studies show that increasing the BiLSTM hidden size to 256 improved average accuracy to 0.7135. Disentangled-attention ablations clarified the role of cross terms: full achieved the top accuracy while p2c_only performed worst, suggesting c2p is more useful than p2c. The lighter variants trained faster with small drops in accuracy for efficiency. Our qualitative analysis indicates that our transformer models tend to be overconfident on entailment when there is high lexical overlap with different logic. But content-to-position and position-to-content cross terms help model be conservative. Overall, we found that choosing the right architecture and being able to model cross-terms interactions are more important than fine-tuning hyperparameters. Our main achievements were building three different architecture models and conducting various types of studies on their performance and behaviors, running targeted ablations. This gives us insights about how to build a good NLP model in the future.

For the primary limitation, our evaluation used single-seed runs, so the results lack confidence intervals and may vary with different seeds or splits.

For future work, we will adopt multi-seed runs and report confidence intervals to quantify variability across seeds and splits.

7 Team Contribution

The Table 6 shows the separate contributions of our teams.

Name	ID	Contribution
Zihan Wu	24372276	Model A design and implementation; Report writing; Ablation study implementation.
Jianyun Yang	24181422	Model C design and implementation; Ablation study implementation; Qualitative Study.
Haoran Yun	24180266	Model B design and implementation; ACL format initialization.

Table 7: Team contribution

References

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2020. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*.

A Attention mechanisms In BiLSTM model

Self-Attention. Each token in a sentence attends to itself to determine its relative importance. For a sequence of BiLSTM hidden states $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_T]$, we compute a scalar attention score for each token as:

$$e_i = \mathbf{w}_s^\top \mathbf{h}_i, \quad (22)$$

$$\alpha_i = \frac{\exp(e_i)}{\sum_{k=1}^T \exp(e_k)}, \quad (23)$$

$$\mathbf{v} = \sum_{i=1}^T \alpha_i \mathbf{h}_i. \quad (24)$$

Here, α_i denotes the normalized importance of token i , and $\mathbf{v}$ is the final sentence representation.

Dot-Product Attention. Given the premise hidden states $\mathbf{H}_P \in R^{m \times d}$ and hypothesis hidden

states $\mathbf{H}_H \in R^{n \times d}$, we first compute a similarity matrix:

$$\mathbf{E} = \mathbf{H}_P \mathbf{H}_H^\top, \quad (25)$$

where each element E_{ij} measures the similarity between the i -th word in the premise and the j -th word in the hypothesis. Attention weights and context representations are computed as:

$$\alpha = \text{softmax}(\mathbf{E}), \quad \mathbf{A}_P = \alpha \mathbf{H}_H, \quad (26)$$

$$\beta = \text{softmax}(\mathbf{E}^\top), \quad \mathbf{A}_H = \beta \mathbf{H}_P. \quad (27)$$

The sentence-level representations are:

$$\mathbf{v}_P = \frac{1}{m} \sum_{i=1}^m \mathbf{A}_{P,i}, \quad \mathbf{v}_H = \frac{1}{n} \sum_{j=1}^n \mathbf{A}_{H,j}. \quad (28)$$

General Attention. This mechanism introduces a learnable linear transformation to improve the similarity measure between premise and hypothesis.

$$\tilde{\mathbf{H}}_H = \mathbf{W}_g \mathbf{H}_H, \quad (29)$$

$$\mathbf{E} = \mathbf{H}_P \tilde{\mathbf{H}}_H^\top, \quad (30)$$

$$\alpha = \text{softmax}(\mathbf{E}), \quad \mathbf{A}_P = \alpha \tilde{\mathbf{H}}_H, \quad (31)$$

$$\beta = \text{softmax}(\mathbf{E}^\top), \quad \mathbf{A}_H = \beta \mathbf{H}_P. \quad (32)$$

The final representations are the mean pooled context vectors:

$$\mathbf{v}_P = \frac{1}{m} \sum_{i=1}^m \mathbf{A}_{P,i}, \quad \mathbf{v}_H = \frac{1}{n} \sum_{j=1}^n \mathbf{A}_{H,j}. \quad (33)$$

Additive (Bahdanau) Attention. Instead of a dot product, this variant uses a feed-forward network to compute attention scores:

$$e_{ij} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{h}_{p_i} + \mathbf{U}_b \mathbf{h}_{h_j}), \quad (34)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^n \exp(e_{ik})}, \quad (35)$$

$$\mathbf{a}_{p_i} = \sum_{j=1}^n \alpha_{ij} \mathbf{h}_{h_j}, \quad (36)$$

$$\mathbf{v}_P = \frac{1}{m} \sum_{i=1}^m \mathbf{a}_{p_i}. \quad (37)$$

A similar computation is applied in reverse (from hypothesis to premise) to obtain $\mathbf{v}_H$.

B Variants In Transformer model

$$\begin{aligned}
\text{full:} & \quad \frac{Q_c K_c^\top + Q_c K_p^\top + Q_p K_c^\top}{\sqrt{d_h}} \\
\text{c2p_only:} & \quad \frac{Q_c K_p^\top}{\sqrt{d_h}} \\
\text{p2c_only:} & \quad \frac{Q_p K_c^\top}{\sqrt{d_h}} \\
\text{c2p_p2c:} & \quad \frac{Q_c K_p^\top + Q_p K_c^\top}{\sqrt{d_h}}
\end{aligned} \tag{38}$$

C Qualitative results

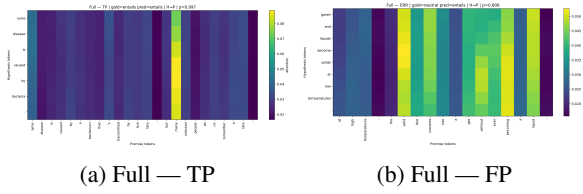


Figure 3: *Full* model: TP vs. FP.

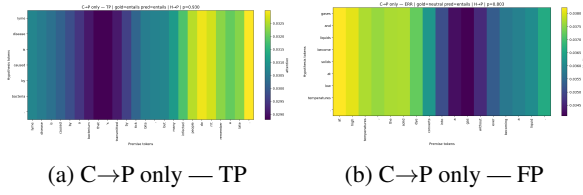


Figure 4: C→P only model: TP vs. FP.